

FAÍSCA



>_

MOMENTO

REVISÃO

>_

Resumo da jornada:

Passamos pela experiência de atuar em um projeto usando **Scrum** e **Kanban**.

>_

Mas um dos grandes pontos de **dificuldade**
enxergado foi o **versionamento...**

>_

o mínimo que preciso saber sobre

VERSIONAMENTO

com Git/Github

>_

Apenas para lembrar...

O que é um **repositório**?

O que é **Git** e **Github**?

>_

GIT



>_

FLUXO INICIAL

até o primeiro commit

>_

Criar um repositório:

Comando: `git init`

>_

Adicionar um arquivo para área de staging:

Comando: `git add .` / `git add nome_arquivo`

>_

Criar commit:

Comando: `git commit -m "mensagem"`

>_

HISTÓRICO

de commits

>_

Ver histórico com infos completas:

Comando: `git log`

>_

Ver histórico com infos reduzidas:

Comando: `git log --oneline`

>_

Ver histórico com infos de quais arquivos foram
alterados:

Comando: `git log --stat`

>_

DESFAZENDO

mudanças

>_

Usei git add num arquivo que eu não queria.

Comando: `git restore --staged arquivo`

>_

Adicionei arquivos pela primeira vez e me arrependi.

Comando: `git clean -f`

>_

BRANCHES

minha maior aliada.

>_

Quero criar uma branch.

Comando: `git checkout -b nome` / `git switch -c`
`nome`

>_

Quero trocar de branch.

Comando: `git checkout nome` / `git switch nome`

>_

Tenho coisas não commitadas (salvas), quero trocar de branch ignorando o que eu fiz.

Comando: `git checkout -f nome`

>_

Quero copiar arquivos de uma branch para outra.

Comando: `git checkout nome -- arquivo`

>_

Quero deletar uma branch local.

Comando: `git branch -D nome`

>_

Renomear uma branch.

Comando: `git branch -M nome`

>_

MERGE

Levar mudanças de uma branch para outra.

>_

Quero trazer mudanças de uma branch para outra.

Comando: `git merge nome`

>_

Por quê neste exemplo o **Git** **não** identificou um
conflito?

>— Quando o Git identifica conflito?

- Se duas branches **modificam** exatamente a mesma linha de um arquivo de forma diferente.
- Branches tem commits diferentes que **alteram** a mesma linha.
- Se um arquivo foi **removido** em uma branch e **alterado** em outra.

>_

Deu conflito, me borrei nas calças e quero parar.

Comando: `git merge --abort`

>_

STASH

Te amo.

>_

Fiz uma porrada de alterações, mas estava na
branch **errada**, quero **salvar** o que fiz e aplicar na
branch certa.

Comando: **git stash**

>_

Visualizar todos os stashes feitos.

Comando: `git stash list`

>_

Aplicar mudanças do primeiro stash da lista.

Comando: `git stash apply`

>_

Aplicar mudanças de um stash específico.

Comando: `git stash apply "stash@{índice}"`

>_

REVERT E RESET

>_

Fiz um commit, mas quero desfazer ele.

Comando: `git reset --hard HEAD~quantidade`

>_

Fiz um commit, comecei a implementar mais coisas,
me arrependi e quero resetar tudo. (se há arquivos
untracked, visualmente não fará diferença)

Comando: `git reset --hard`

>_

Fiz um commit, comecei a implementar mais coisas,
me arrependi quero resetar tudo, mas o que eu fiz
eu quero manter.

Comando: `git reset --mixed HEAD~quantidade`

>_

Fiz um commit, mas me arrependi.

Comando: `git revert hash`

>_

REBASE

>_

Tenho a branch **X**, criei a branch **Y** a partir de **X**, fiz commits em **Y**, mas também foram feitos commits em **X** enquanto eu mexia em **Y**.

>_

Comando: git rebase nome

>_

Fiz vários commits, mas quero juntar tudo em só.

Comando: `git rebase --interactive HEAD~3`

>_

COMANDOS

ESPECIAIS

>_

Quero trazer um commit específico de uma branch
para outra.

Comando: `git cherry-pick hash`

>_

Fiz vários `commits`, mas em algum momento
`quebrou` tudo, o que posso fazer?

>_

Posso navegar em todos os `commits` e ver o que foi
feito... Mas se eu tiver muitos `commits`, vira um
processo `lento` e `trabalhoso`...

>_

Comando:

git bisect start

git bisect good hash

git bisect bad hash

git bisect reset

>_

MÃO NA MASSA



Objetivo: Criar uma receita de bolo.

Regras:

- Formar duplas.
- Criar um repositório.
- Criar branch main com um arquivo `receita.txt`.
- Trabalhando de forma paralela em cima do **MESMO** arquivo, uma pessoa será responsável por `adicionar` os ingredientes, enquanto a outra `adiciona` o método de preparo.
- Cada pessoa deve criar uma branch para si.
- No final, **juntar** as branches na `main`.